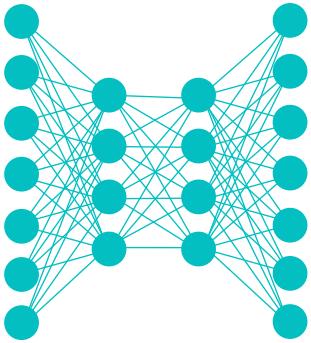
Lecture Notes for

Deep Learning

Practical and Efficient Transformers

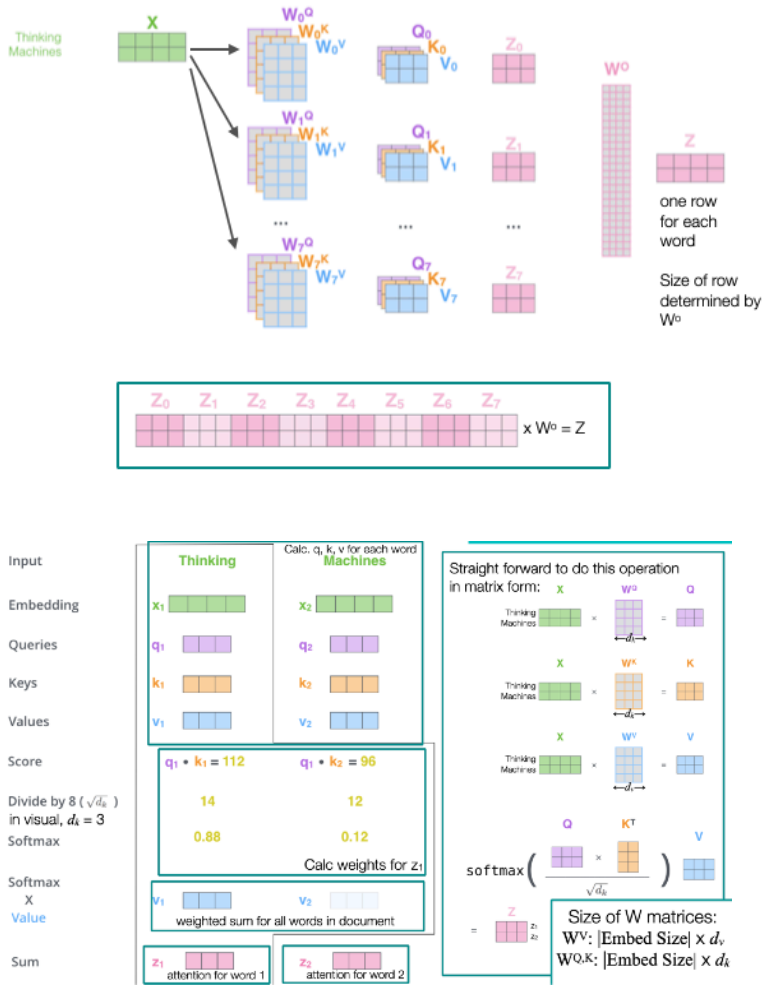


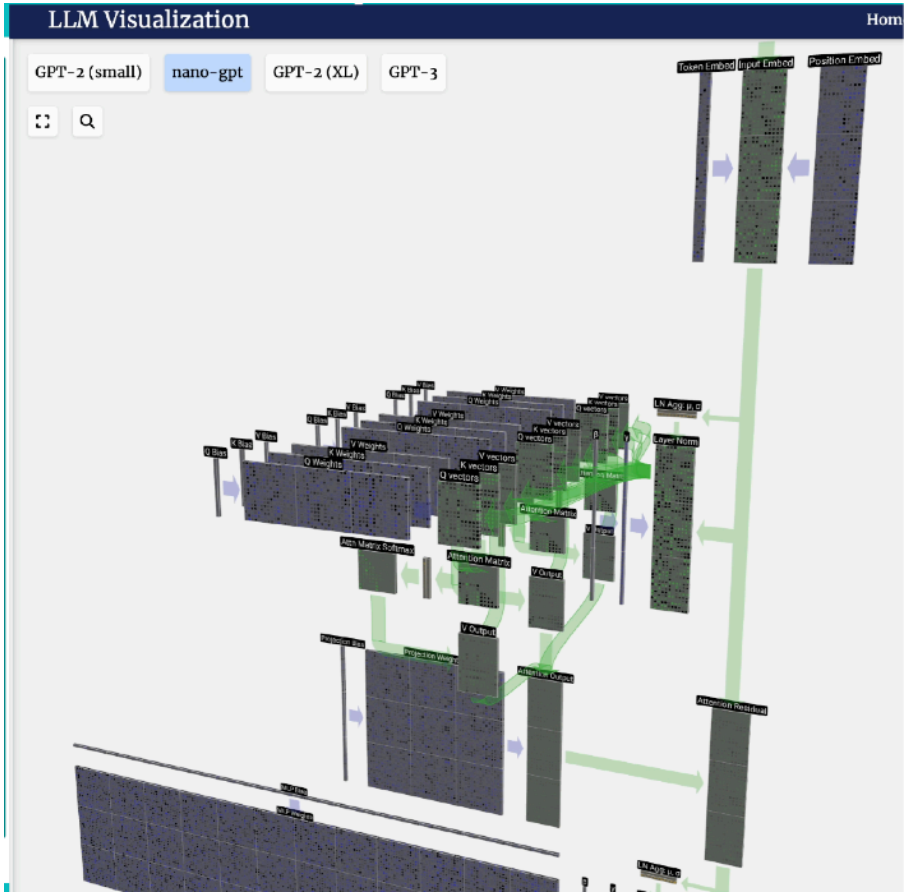
Logistics and Agenda

- Logistics
  - Lab and grading
  - Next lectures: Lots of Paper Presentations!
- Agenda
  - Efficient Attention
  - Positional Encoding
- Next Time:
  - Popular encoder models (maybe start today)

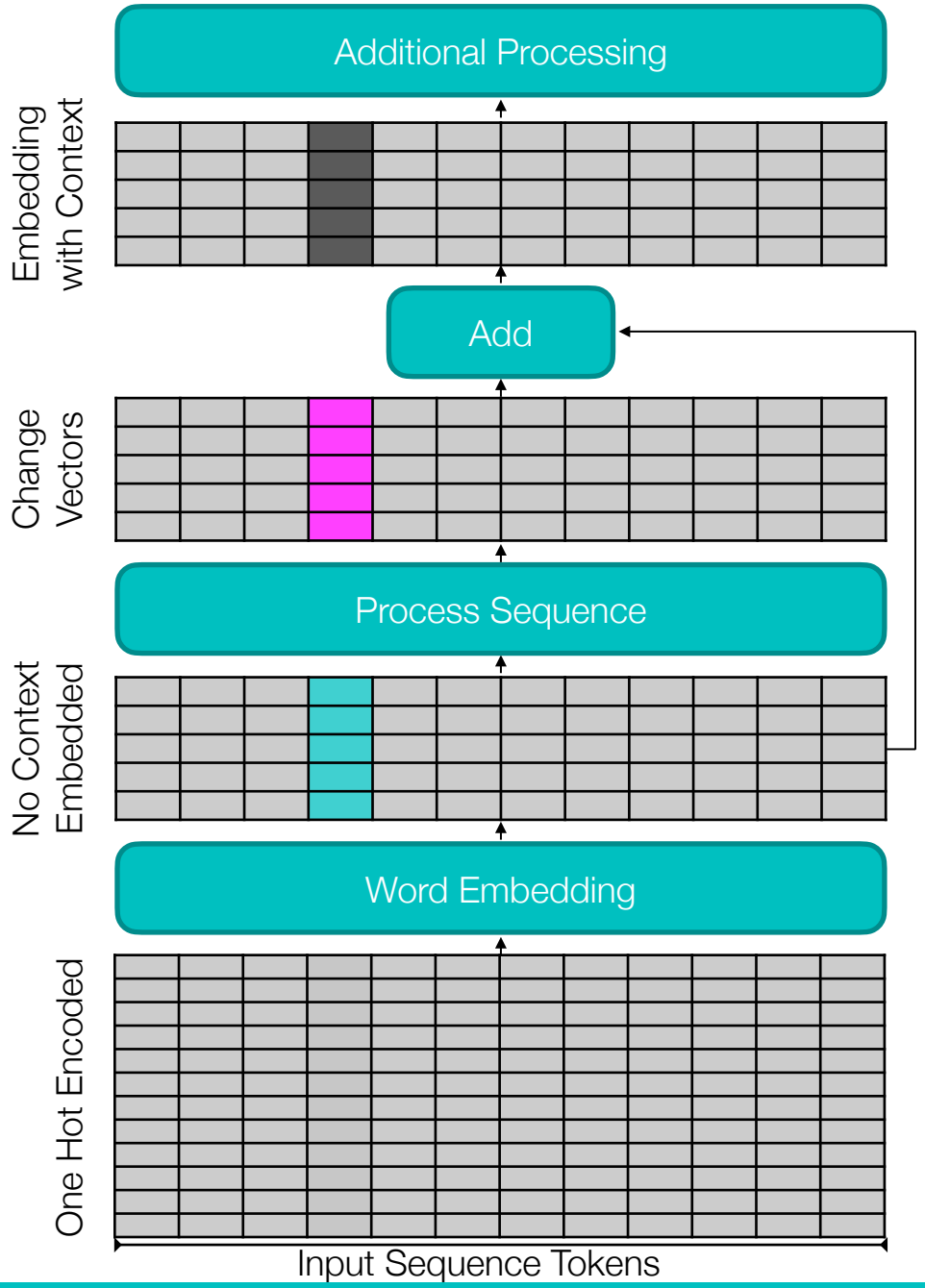
Last Time: Transformers

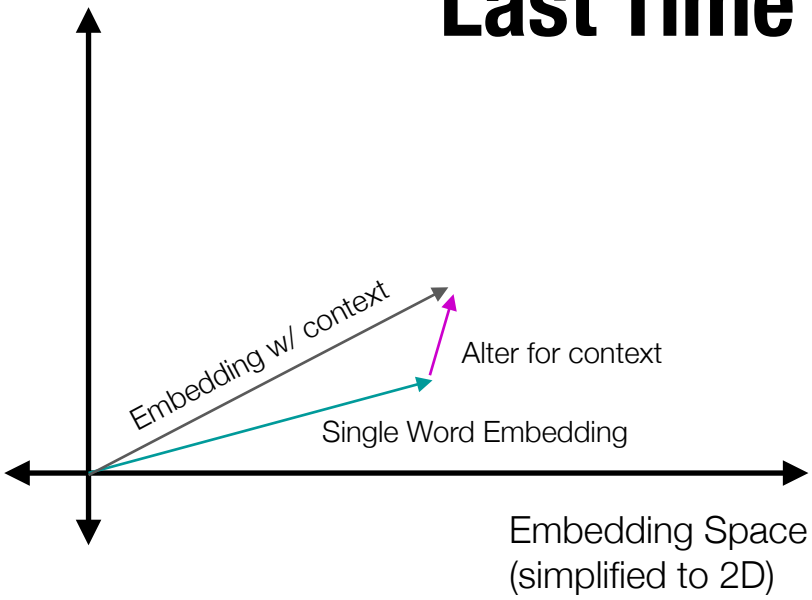
Transformer: Multi-headed Attention





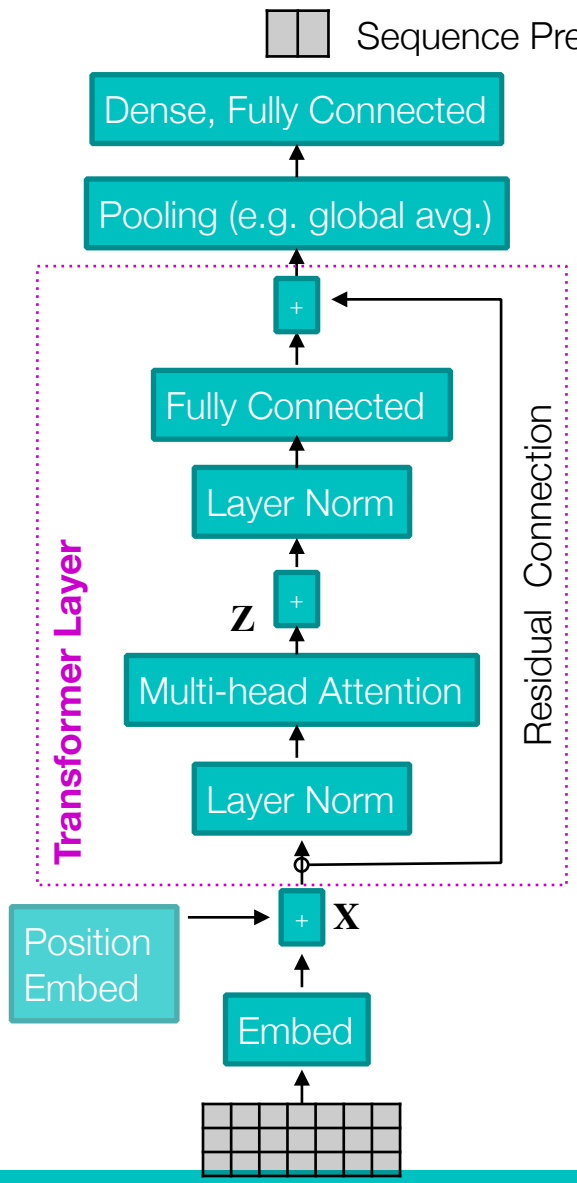
Last Time

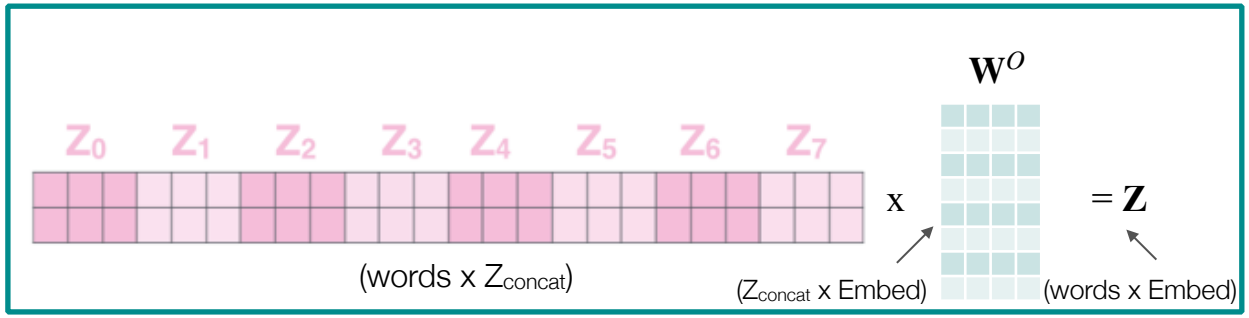




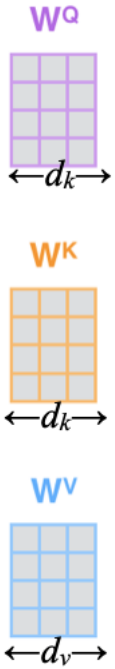
- Transformer is tasked with generating the “alter context” vector
- “alter” vector is in a high dimensional space (word embedding space)
- Calculated from lower dimension attention space
  - Look at all words in the sequence, take weighted sum
- Expand back to original embedding space for addition
- Residual also helps with gradient

All together: Transformer Review






```
tf.keras.layers.MultiHeadAttention(
    num_heads,      (Number of heads Z1-Z7)
    key_dim,        (size of query/key dk)
    value_dim,      (size of each dv)
    output_shape,   (control size of Wo)
     …
)
```



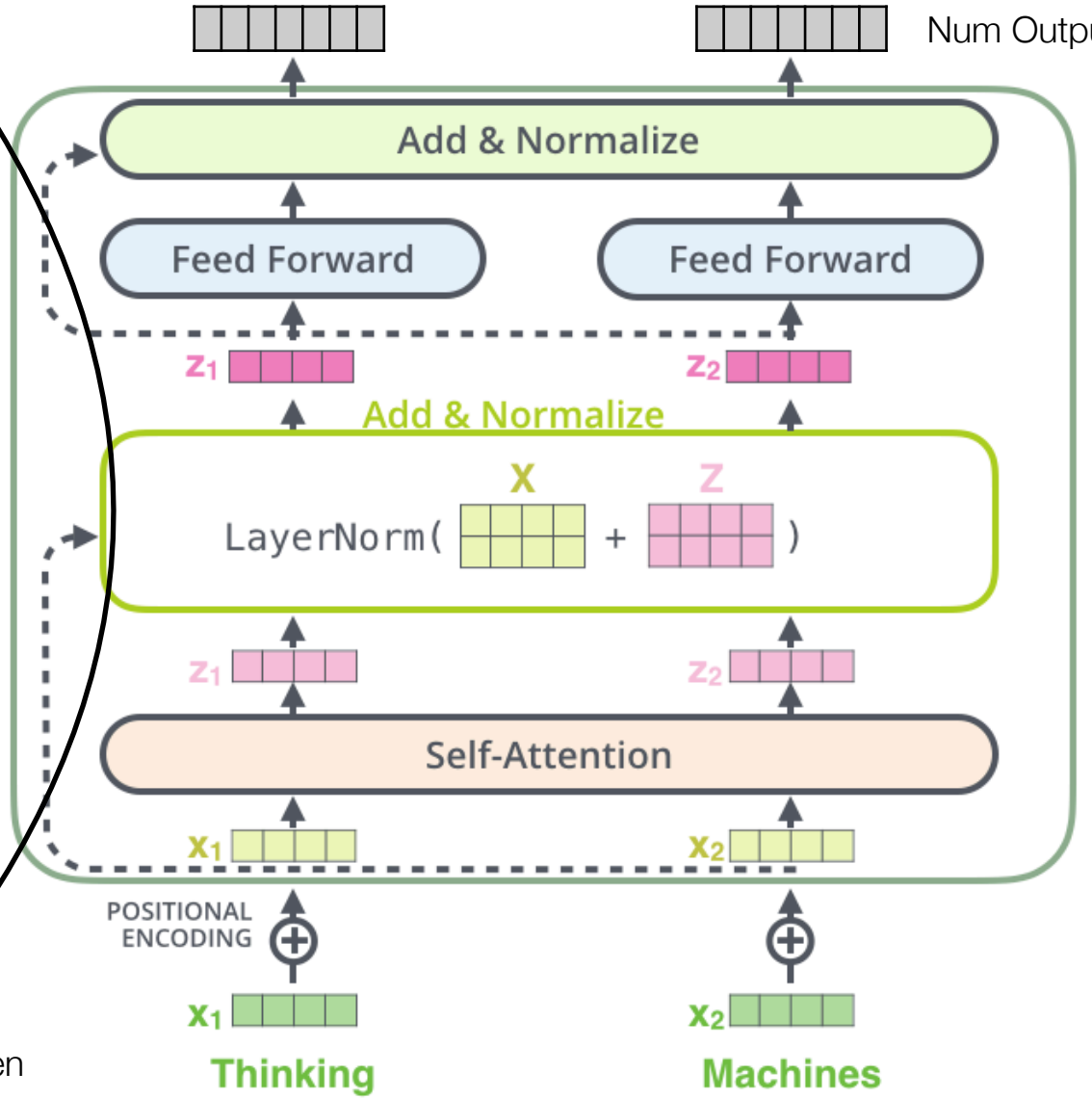
Transformer for Sequence Classification

Num Outputs is Same as Inputs

Use for Prediction about entire sequence, ignore other outputs!

Only back propagate loss through this last token.

Prepend all sequences with same token [CLS]

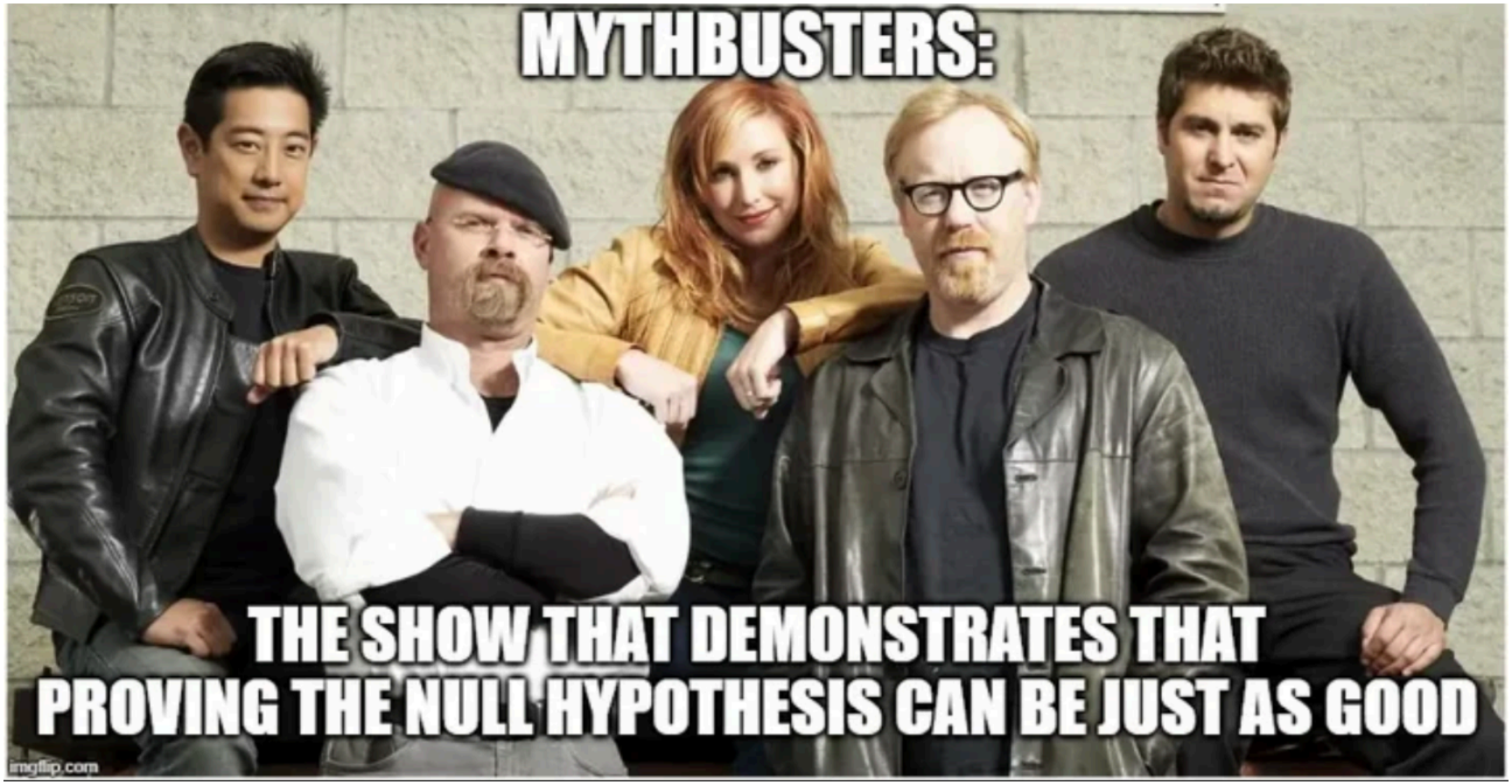


Each output is the attended sum of every input.

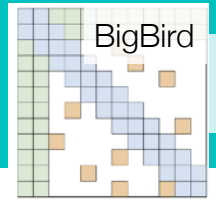
x_{class}

"class" token

Altering Self-Attention



Attention Efficiently

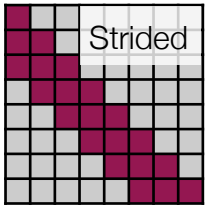


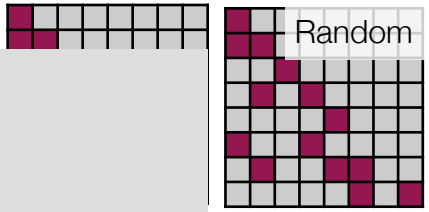
Partial Global + Rand + Strided
Zaheer et al., **BigBird**, NeurIPS 2021

Naive Implementation:

- Computation: $O(L^2 \cdot d)$
- Memory: $O(L^2 + L \cdot d)$
- Quadratic with sequence length

One idea: limit non-zero values of $\mathbf{Q} \cdot \mathbf{K}^T$
Need to define sparsity before computation





Another idea: change softmax, to allow associative rule application

$$\underbrace{\left(\mathbf{Q} \cdot \mathbf{K}^T\right) \cdot \mathbf{V}}_{O(L^2 \cdot d)} = \underbrace{\mathbf{Q} \cdot \left(\mathbf{K}^T \cdot \mathbf{V}\right)}_{O(L \cdot d^2)}$$

KV can be cached

Efficient Implementation:

- Computation: $O(L \cdot d^2)$
- Memory: $O(d^2 + L \cdot d)$
- Linear with sequence length

but we need a function that satisfies

$$f\left(\mathbf{Q} \cdot \mathbf{K}^T\right) = f(\mathbf{Q}) \cdot f(\mathbf{K}^T) \quad \textit{Linear Attention}$$

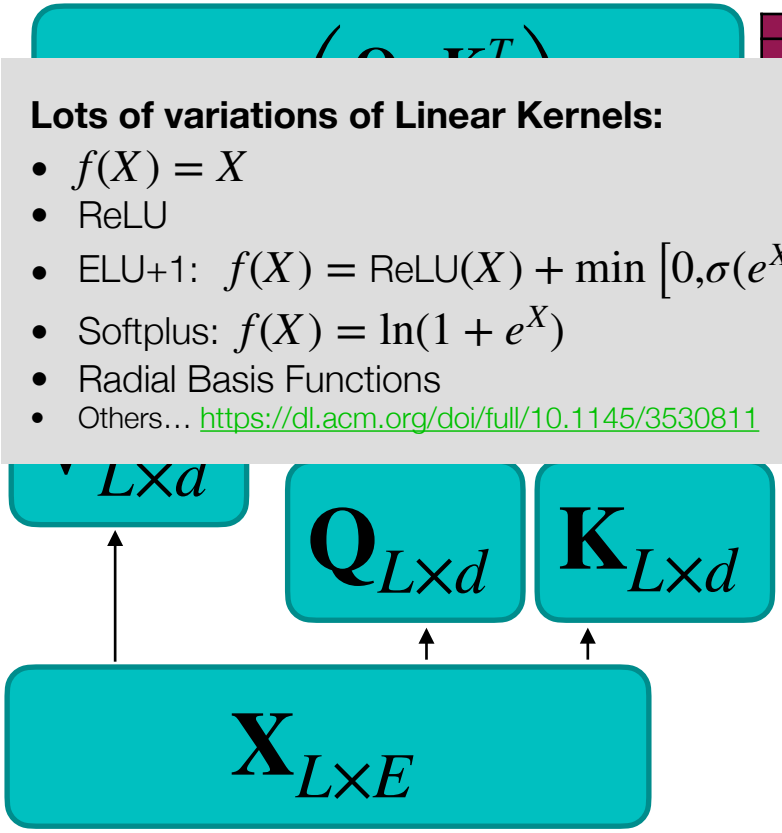
One function: softmax along rows and columns

$$\text{softmax}(\mathbf{Q}) \cdot \left(\text{softmax}(\mathbf{K})^T \cdot \mathbf{V}\right)$$

Katharopoulos et al., **Trans are RNNs**, ICLR 2021

Lots of variations of Linear Kernels:

- $f(X) = X$
- ReLU
- ELU+1: $f(X) = \text{ReLU}(X) + \min\left[0, \sigma(e^X - 1)\right]$
- Softplus: $f(X) = \ln(1 + e^X)$
- Radial Basis Functions
- Others… https://dl.acm.org/doi/full/10.1145/3530811



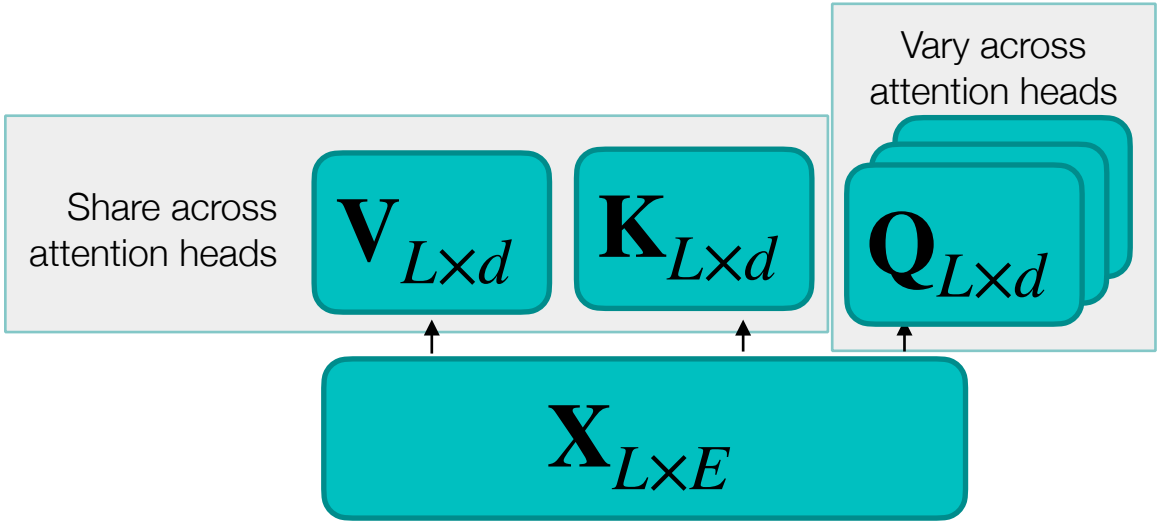
Another: $$\frac{\mathbf{Q}}{\|\mathbf{Q}\|} \cdot \left(\frac{\mathbf{K}}{\|\mathbf{K}\|}^T \cdot \mathbf{V}\right) \rightarrow \frac{\mathbf{Q} \cdot \mathbf{K}^T}{\|\mathbf{Q}\| \|\mathbf{K}\|} \cdot \mathbf{V}$$

same as cosine similarity

Mongaras, Dohm, and Larson, **Cottention**, CC 2025

$\boldsymbol{E}$: token embedding size, $\boldsymbol{L}$: sequence length, $\boldsymbol{d}$: transformer dimension

Multi-query Attention (MQA)

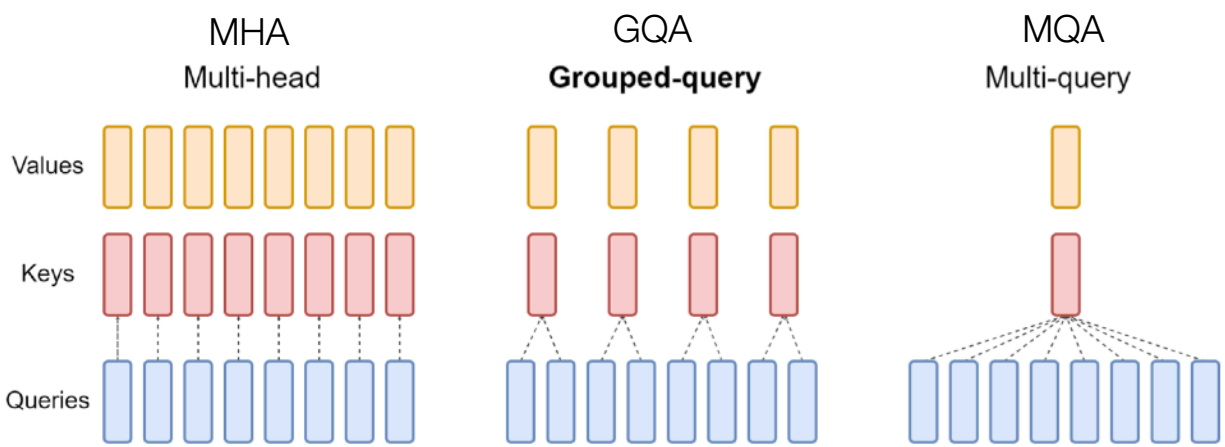


- (-) Slight drop in accuracy for various tasks
- (+) Allows larger transformer feed forward layers
- (+) larger context lengths fit in GPU memory
- (-) No speed up for distributed compute as K, V are copied

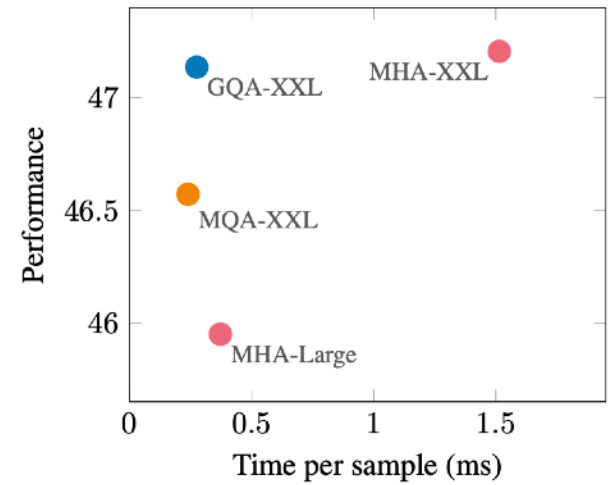
- Vanilla transformer can store V and K on SRAM of GPU, then just load in Q from high-bandwidth memory (HBM)
  - memory transfer is critical bottleneck for GPU, so you get a huge speed up
- For methods that can calculate $\mathbf{Q}_i \cdot \left(\mathbf{K}^T \cdot \mathbf{V}\right)$, the entire $\mathbf{K}^T \cdot \mathbf{V}$ matrix can be precomputed (*may not fit in SRAM for long sequences*)

Shazeer, Fast Transformer Decoding, ArXiV (2019)

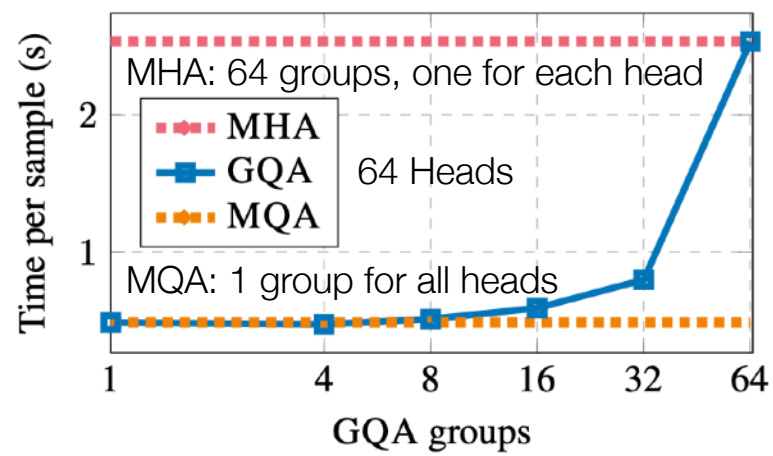
Group Query Attention



matrices needed per head, number of heads = unique query matrices



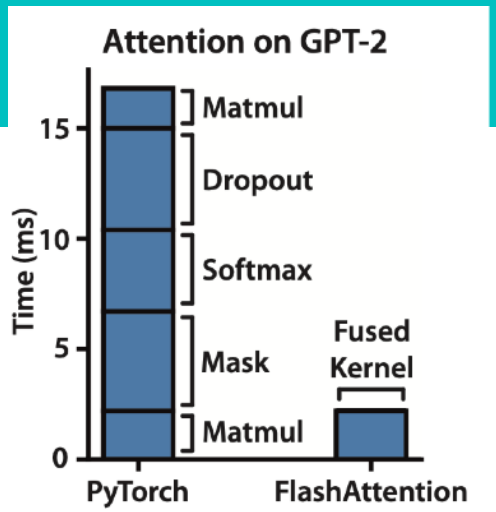
- Can take advantage of distributed computation, parallelize across groups for unique K, V
- Easy to tradeoff performance of MHA with compute of MQA

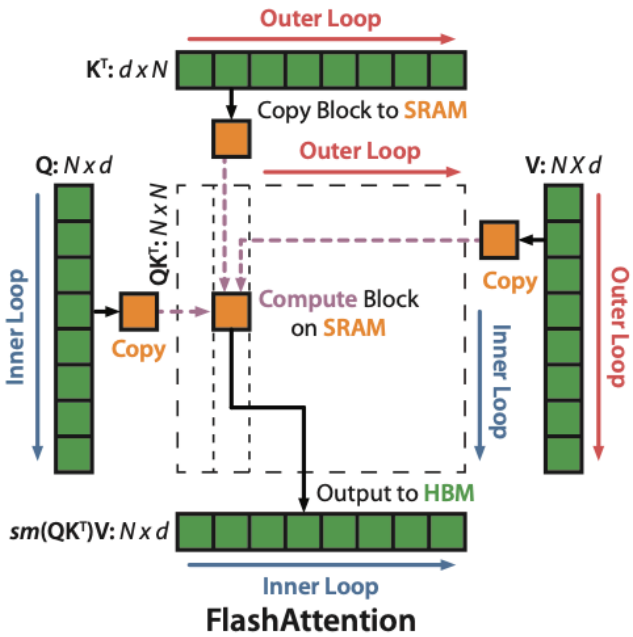


Ainslee et al., GQA, ArXiV (2023)

Flash Attention

- Calculate attention in tiles (local compute)
  - Requires calculation and saving additional variables in each tile
- During tile aggregation, scale the variables properly to get exact softmax across tiles (distributed softmax)
- Tile calculation is a shader function, massive speed up on a GPU
- Back-prop: Only save the attention output and recompute it for back propagation to save memory
  - similar to gradient checkpointing, this adds compute *in training* but saves memory
- **Flash Attention 2:**
  - Some small improvements to matrix multiplications
  - Added support for GQA (big speed ups)
  - Flash Attention becomes 9x faster than normal attention for both training and inference





Dao et al., FlashAttention, NeurIPS 2022

Distributing Softmax in Tiles

$$\mathbf{x} = \mathbf{q}_t \cdot \mathbf{K}_{1:t}^T$$

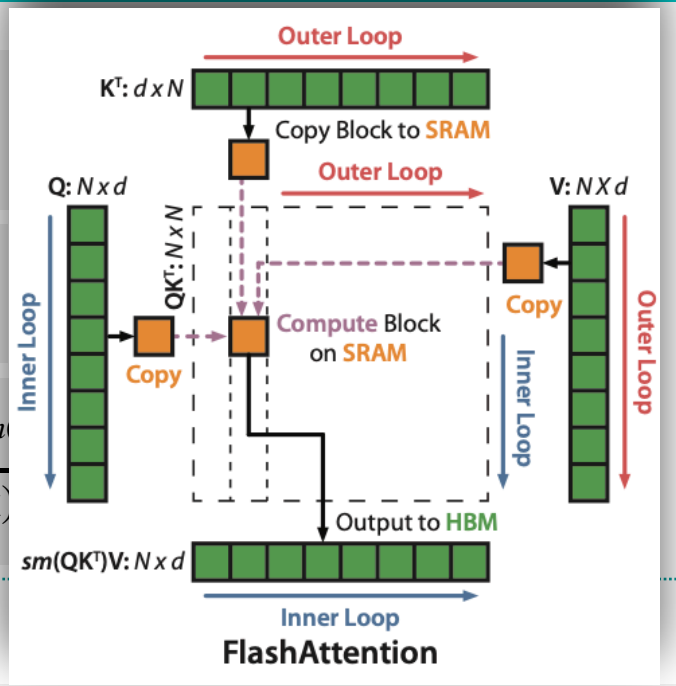
Need softmax($\mathbf{x}$)

$$\mathbf{x} = [a, b, c, d]$$

$$m(\mathbf{x}) = \max\left([a,\right.$$

$$f(\mathbf{x}) = \left[e^{a-m(\mathbf{x})}, e^{b-m(\mathbf{x})}, e^{c-m(\mathbf{x})}, e^{d-m(\mathbf{x})}\right]$$

$$\text{softmax}(\mathbf{x}) = \frac{f(\mathbf{x})}{l(\mathbf{x})} = \left[\frac{e^{a-m(\mathbf{x})}}{l(\mathbf{x})}, \frac{e^{b-m(\mathbf{x})}}{l(\mathbf{x})}, \frac{e^{c-m}}{l(\mathbf{x})}\right.$$



Regular Softmax Calculation

$$\mathbf{x}^{(1)} = [a, b] \quad f(\mathbf{x}^{(1)}) = \left[e^{a-m(\mathbf{x}^{(1)})}, e^{b-m(\mathbf{x}^{(1)})}\right]$$

$$\mathbf{x}^{(2)} = [c, d] \quad f(\mathbf{x}^{(2)}) = \left[e^{c-m(\mathbf{x}^{(2)})}, e^{d-m(\mathbf{x}^{(2)})}\right]$$

$$m(\mathbf{x}) = \max\left(m(\mathbf{x}^{(1)}), m(\mathbf{x}^{(2)})\right)$$

Need to track this

$$s_i = e^{m(\mathbf{x}^{(i)})-m(\mathbf{x})}$$

Slightly more FLOPS, but better utilization of parallelism

$$f(\mathbf{x}) = \left[s_1 \cdot f\left(\mathbf{x}^{(1)}\right), s_2 \cdot f\left(\mathbf{x}^{(2)}\right),\right]$$

$$l(\mathbf{x}) = s_1 \cdot l\left(\mathbf{x}^{(1)}\right) + s_2 \cdot l\left(\mathbf{x}^{(2)}\right)$$

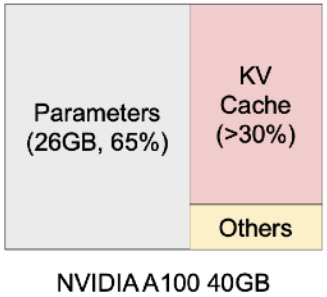
Combine distributed ops:

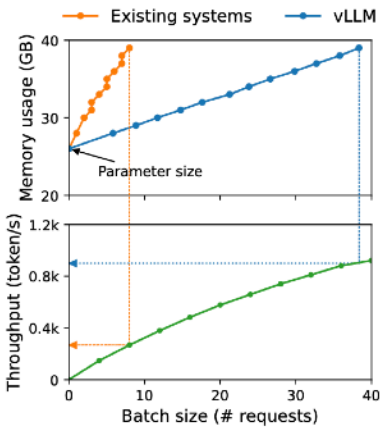
$$\text{softmax}(\mathbf{x}) = \frac{f(\mathbf{x})}{l(\mathbf{x})}$$

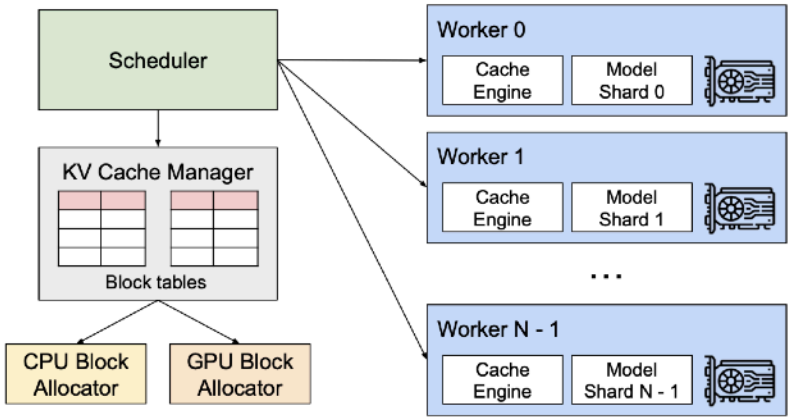
Distributed Softmax

Paged Attention

- Straight up computer science issue in linear attention
- For large sequences, KV operation is fragmented across memory and compute
  - Hard to cache KV accesses
- Page KV cache in memory aligned blocks similar to virtual memory on OS
  - Vastly reduces fragmentation and speeds up accesses

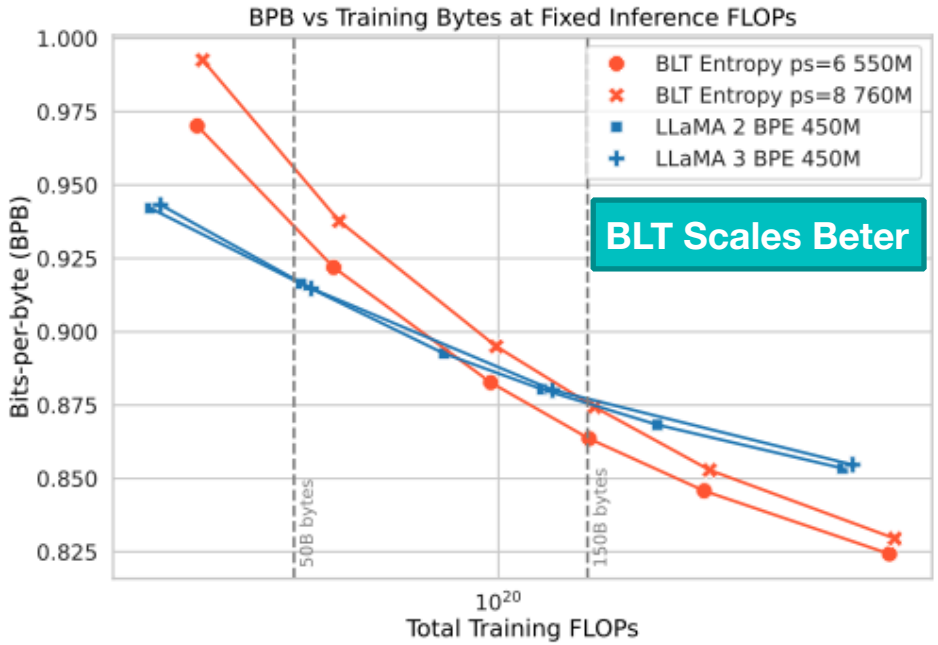


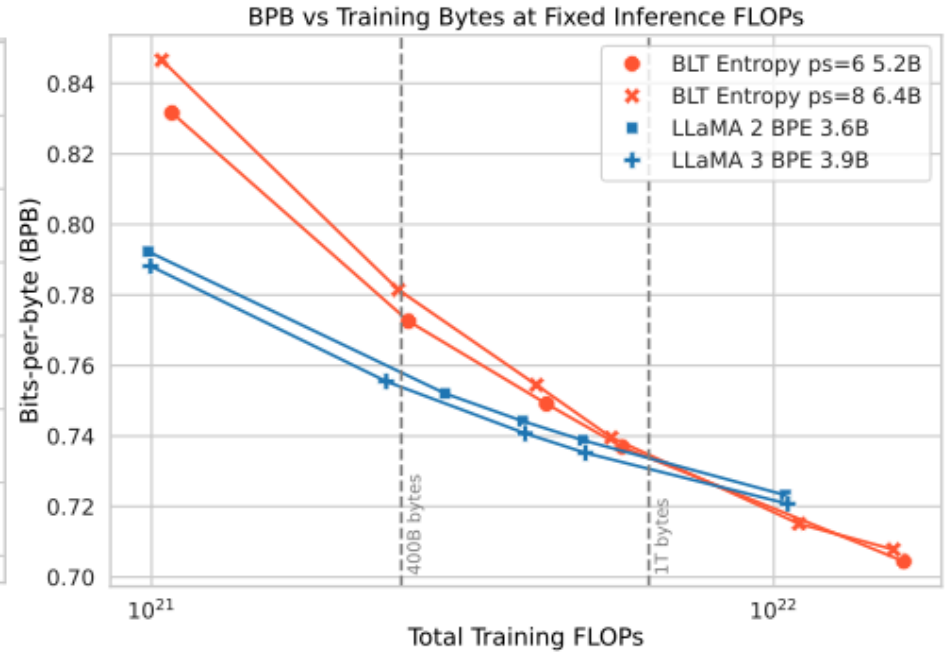




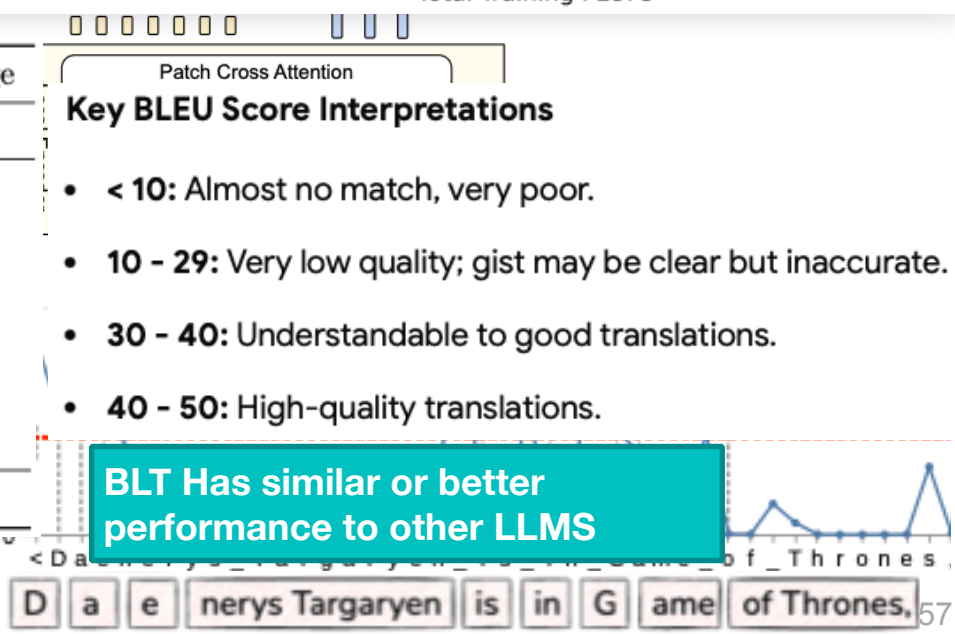
- Store the KV cache in non-contiguous blocks for better parallelized access.
- Supports parallelized sequence access with fewer misses.

Kwon et al., Efficient Memory, SOSP (2023)





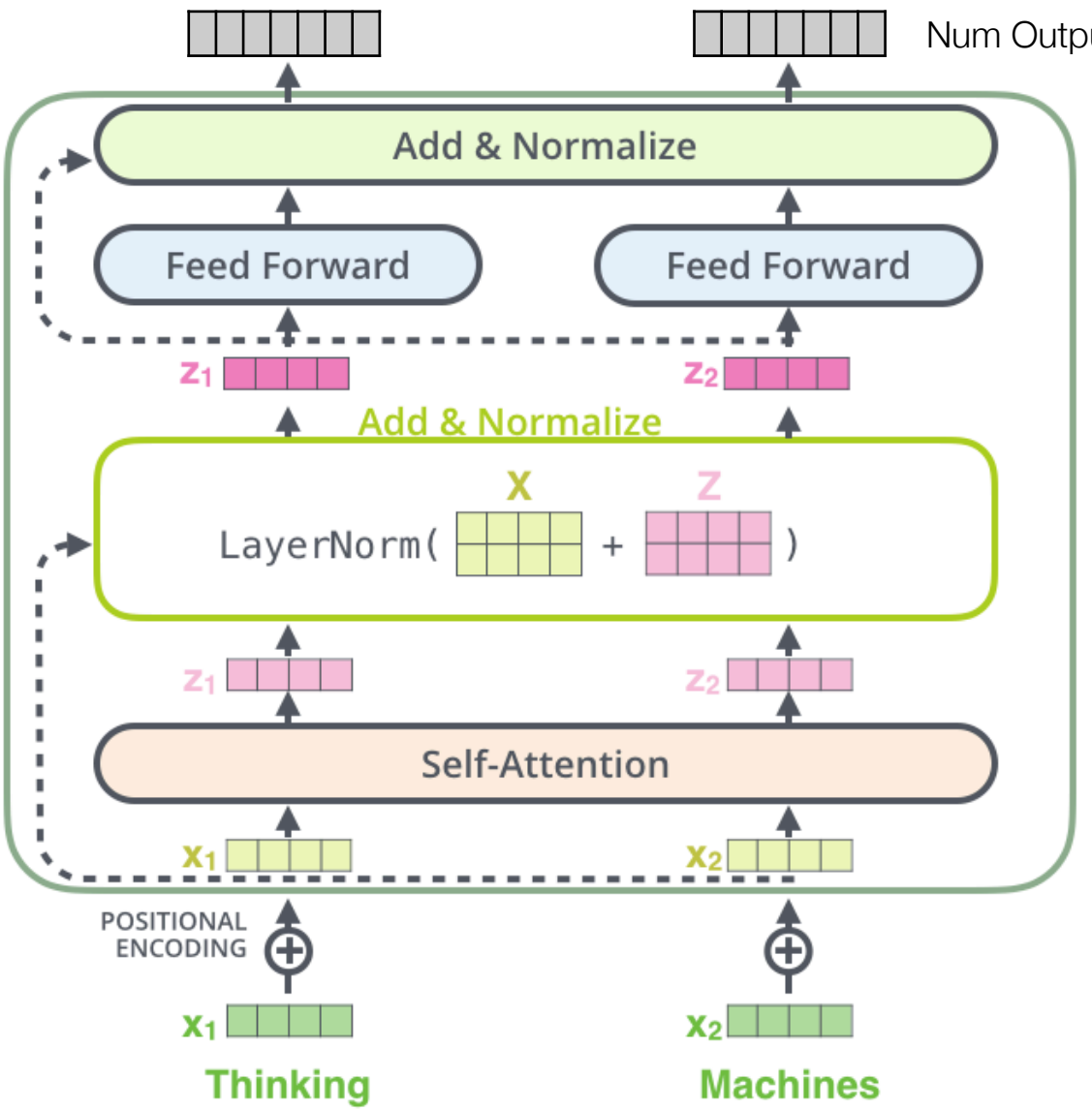
Language	Language → English		English → Language	
	Llama 3	BLT	Llama 3	BLT
Arabic	22.3	24.6	10.4	8.8
German	41.3	42.0	29.8	31.2
Hindi	20.7	20.9	7.8	7.2
Italian	34.0	33.9	24.4	26.2
Vietnamese	31.2	31.0	28.4	23.7
Thai	17.9	18.1	10.5	7.7
… Many other languages …				
Overall Average	12.1	**14.0**	5.9	**6.4**



Pagnoni et al., BLT: Patches Scale Better than tokens, ACL (2025)

Positional Encoding

Transformer for Sequence Classification



Num Outputs is Same as Inputs

Do these outputs change, if the input sequence changes order?

The order of vectors will change, but not the values of each vector…

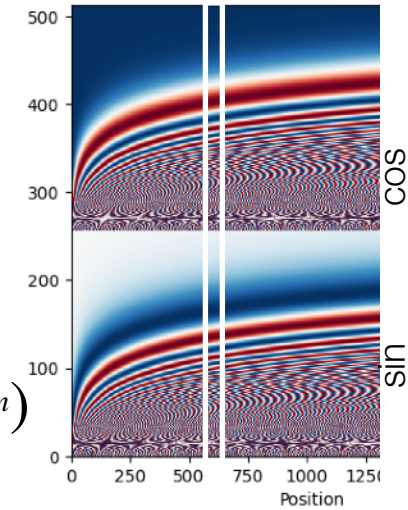
$$\text{softmax}\left(\frac{\mathbf{Q}\cdot\mathbf{K}^T}{\sqrt{d_k}}\right)\cdot\mathbf{V}$$

$$\mathbf{V}_{(1:L)} \quad \mathbf{Q}_{(1:L)} \quad \mathbf{K}_{(1:L)}$$

$$\mathbf{X}_{(1:L)}$$

Transformer: First Positional Encoding

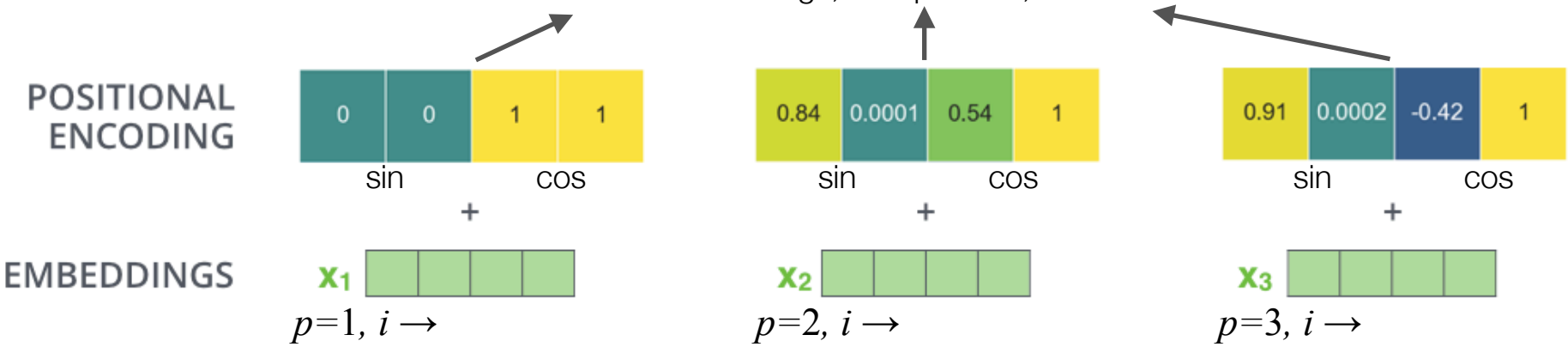
- Objective: add notion of position to embedding
- Attempt in paper: add sin/cos to embedding



p: in sequence
d_m: 1/2 dim of embed
i = index in vector

$$PE_{(p,i\in 0\ldots d_m-1)} = \sin(p/10000^{i/d_m})$$
$$PE_{(p,i\in d_m\ldots 2d_m)} = \cos(p/10000^{(i-d_m)/d_m})$$

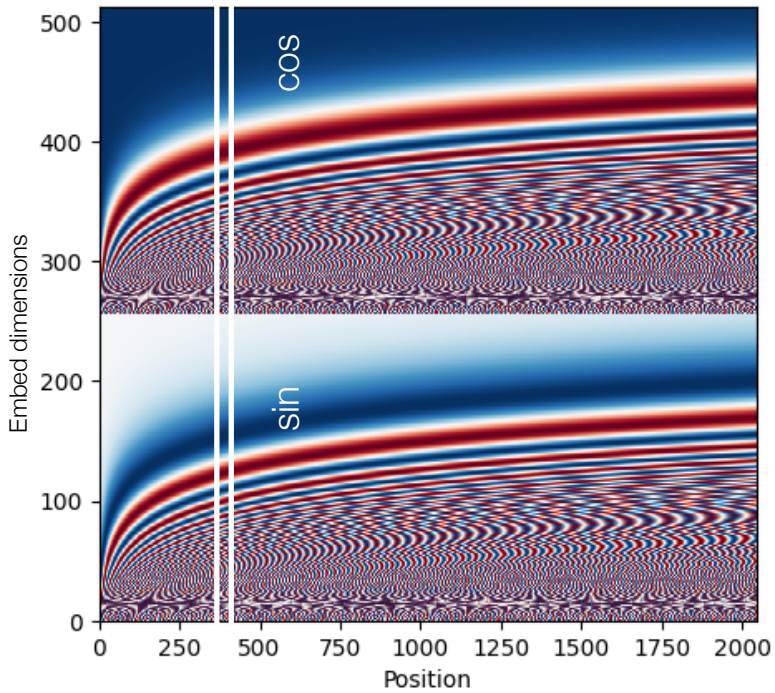
Now use the new embeddings, with position, into transformer architecture

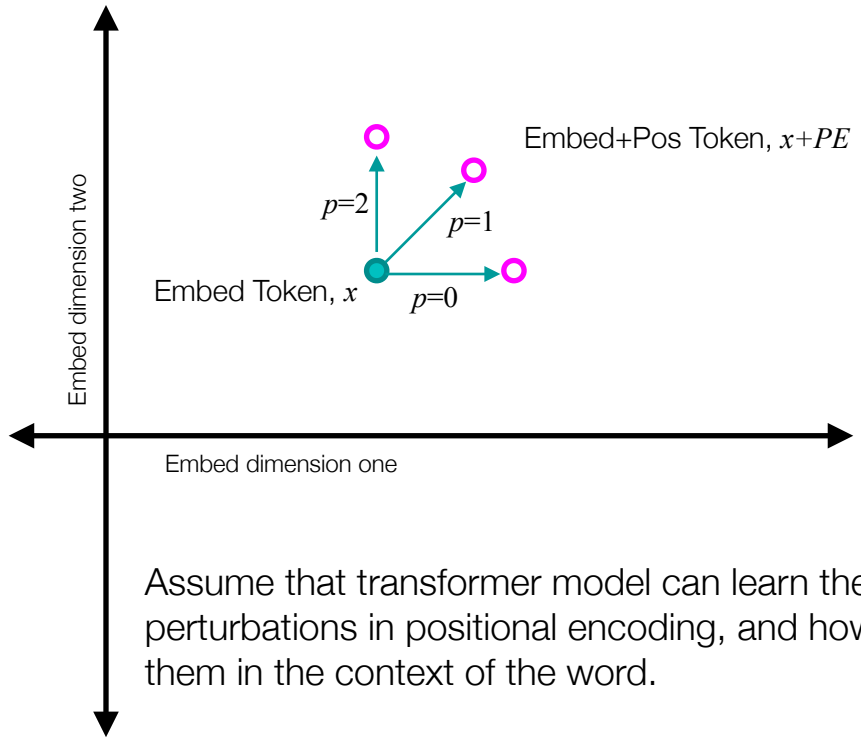


Hypothesis: Now the word proximity is encoded in the embedding matrix, with other pertinent information. Well, it does help… so it could be true that this is a good way to do it.

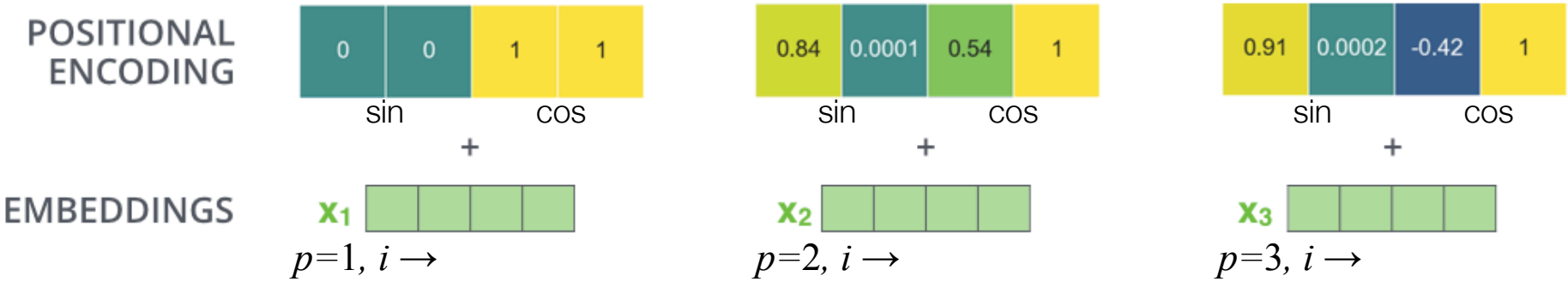
Excellent Blog on Transformers: http://jalammar.github.io/illustrated-transformer/

Positional Intuition, Geometrically





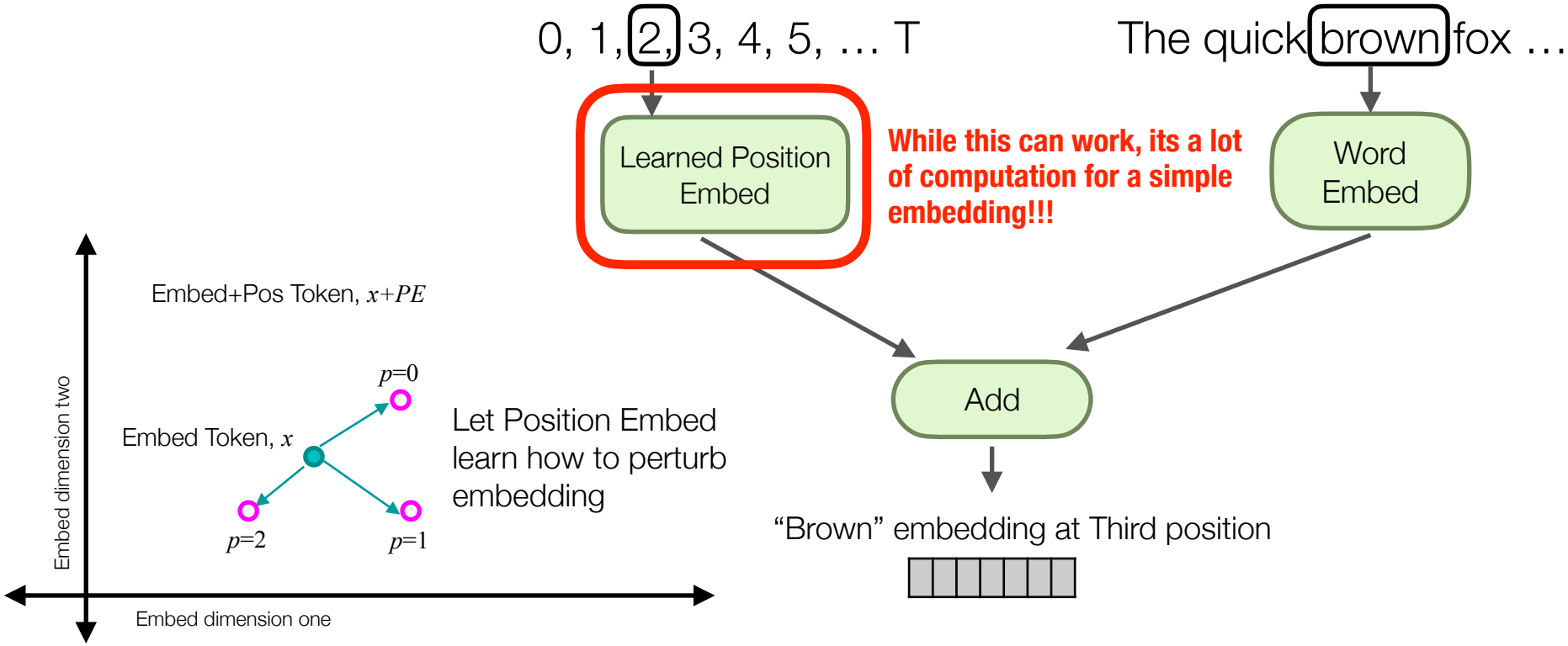
Assume that transformer model can learn the small perturbations in positional encoding, and how to use them in the context of the word.



Transformer: Positional Embedding

- Object
- Attemp
- **But co**

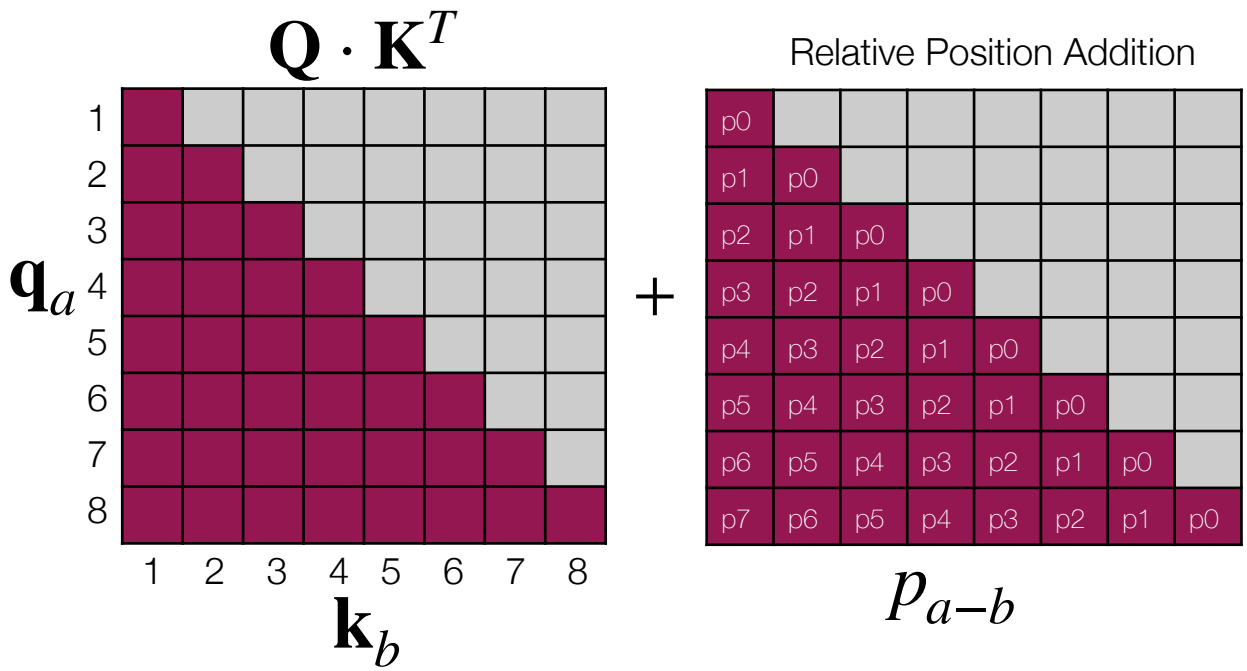
What if, instead of perturbing each embedding based on position, we perturb the multiplication of two embeddings, **based on their relative position?**



Excellent Blog on Transformers: http://jalammar.github.io/illustrated-transformer/

Relative Positional Encoding

- Don't encode raw position, encode relative position between $\mathbf{q}_a$ and $\mathbf{k}_b$ at position a and b
- practically, attention → softmax($\mathbf{Q} \cdot \mathbf{K}^T + p_{a-b}$)



- (+) Nicely structured relative position information
- (-) Lots more memory
- (-) Couples position and attention (harder to distribute ops)

- **How might we still encode relative position, without all the overhead?**